

Notes on Streams and Lazy Evaluation in Scheme (SICP)

Pushpendra Uikey

1. Introduction

In functional programming, a **stream** is a potentially infinite sequence of elements that are computed *lazily*. Rather than generating all elements upfront, a stream computes each element only when required. This enables working with infinite data structures like all integers or the Fibonacci sequence.

2. Delay and Force Mechanism

The foundation of lazy evaluation is built on **delay** and **force**.

```
(define-syntax delay
  (syntax-rules ()
    ((delay expr) (lambda () expr))))

(define (force promise) (promise))
```

- **delay**: Wraps an expression `expr` inside a zero-argument procedure (a *promise*), delaying its evaluation.
- **force**: Forces the evaluation of the delayed expression by calling the promise.

This mechanism prevents premature computation of stream tails.

3. Stream Construction and Access

Streams are built using `cons-stream`, a macro that constructs a pair whose tail is delayed.

```
(define-syntax cons-stream
  (syntax-rules ()
    ((cons-stream a b) (cons a (delay b)))))
```

Supporting functions:

```
(define (stream-car s) (car s))
(define (stream-cdr s) (force (cdr s)))
(define the-empty-stream '())
(define (stream-null? s) (eq? s the-empty-stream))
```

Explanation:

- `stream-car` returns the first element.
- `stream-cdr` forces the delayed computation of the rest.
- `stream-null?` checks if the stream is empty.

Thus, only the *head* of the stream is evaluated immediately; the tail is deferred.

4. Stream Mapping and Filtering

Operations like `map` and `filter` are defined recursively over streams.

```
(define (stream-map proc s)
  (if (stream-null? s)
      the-empty-stream
      (cons-stream (proc (stream-car s))
                    (stream-map proc (stream-cdr s)))))

(define (stream-filter pred s)
  (cond ((stream-null? s) the-empty-stream)
        ((pred (stream-car s))
         (cons-stream (stream-car s)
                       (stream-filter pred (stream-cdr s))))
        (else (stream-filter pred (stream-cdr s)))))
```

Explanation:

- `stream-map`: applies a function to each stream element lazily.
 - `stream-filter`: constructs a new stream containing only elements satisfying the predicate.
-

5. Enumerating Streams

Finite or infinite streams can be generated recursively.

```
(define (stream-enumerate-interval low high)
  (if (> low high)
      the-empty-stream
      (cons-stream low
                    (stream-enumerate-interval (+ low 1) high))))
```

`(stream-enumerate-interval 1 5)` produces a finite stream `[1, 2, 3, 4, 5]`.

6. Infinite Streams

(a) Stream of Integers

```
(define (integers-starting-from n)
  (cons-stream n (integers-starting-from (+ n 1))))
(define integers (integers-starting-from 1))
```

This creates an infinite stream of positive integers. Using `stream-ref` we can access the n th element without computing all previous ones eagerly.

```
(define (stream-ref s n)
  (if (= n 0)
      (stream-car s)
      (stream-ref (stream-cdr s) (- n 1))))
```

Example:

```
(stream-ref integers 0) => 1
(stream-ref integers 100) => 101
```

(b) Filtering Multiples

```
(define (divisible? x y) (= (remainder x y) 0))
(define nd7 (stream-filter (lambda (x)
  (not (divisible? x 7))) integers))
(stream-ref nd7 1000)
```

This defines a stream of all integers not divisible by 7. Even though the stream is infinite, only as many elements as needed are computed.

7. Infinite Fibonacci Stream

```
(define (fibgen a b)
  (cons-stream a (fibgen b (+ a b))))
(define fibs (fibgen 0 1))
```

Each Fibonacci number is generated from the previous two lazily. Example:

```
(stream-ref fibs 3) => 2
(stream-ref fibs 10) => 55
(stream-ref fibs 100) => 354224848179261915075
```

8. Summary

- **delay/force**: mechanism for deferred evaluation.
- **cons-stream**: constructs a stream with a delayed tail.
- **stream-car** / **stream-cdr**: retrieve head and lazily compute tail.
- **stream-map**, **stream-filter**: define infinite transformations without explicit loops.
- Streams allow handling of *infinite sequences* using finite memory, leveraging lazy evaluation.

9. Macros: define-syntax and syntax-rules

In Scheme, **define-syntax** is used to define **macros**—compile-time transformations that generate new code patterns before evaluation. They allow creating new syntactic constructs, i.e., custom keywords that behave like language features.

(a) define-syntax

The general form is:

```
(define-syntax <name>
  <transformer-spec>)
```

Here, **<name>** becomes a new syntactic keyword, and **<transformer-spec>** determines how expressions using this keyword are transformed before execution.

(b) syntax-rules

syntax-rules provides a pattern-matching mechanism to define hygienic macros.

```
(define-syntax <name>
  (syntax-rules (<literals>)
    ((<pattern_1>) <template_1>)
    ((<pattern_2>) <template_2>)
    ...
  ))
```

- Each **pattern** specifies how a call to the macro should look.
 - Each **template** specifies the code that replaces the pattern.
 - The system automatically substitutes variables from the pattern into the template.
-

(c) Example: delay

```
(define-syntax delay
  (syntax-rules ()
    ((delay expr) (lambda () expr))))
```

Explanation:

- The pattern `(delay expr)` matches any expression of the form `(delay <something>)`.
 - The template expands it into `(lambda () expr)`, i.e., a zero-argument procedure that returns the value of `expr` when called.
 - Thus, `delay` introduces *lazy evaluation* by postponing computation.
-

(d) Example: cons-stream

```
(define-syntax cons-stream
  (syntax-rules ()
    ((cons-stream a b) (cons a (delay b)))))
```

Explanation:

- This pattern `(cons-stream a b)` matches two arguments.
 - The template builds a normal pair using `cons`, but wraps the second element `b` with `delay`, making the tail of the stream lazy.
 - As a result, `cons-stream` behaves like a **lazy cons**, evaluating the first element immediately but deferring the computation of the rest.
-

(e) Summary

- `define-syntax` creates new syntactic constructs.
- `syntax-rules` defines how those constructs expand via pattern matching.
- Together, they allow building domain-specific abstractions (like streams) without changing the core language.